

WEB APPLICATION DEVELOPMENT

LAB 6 – HOMEWORK

EXERCISE 5: AUTHENTICATION FILTER

Task 5.1: Create AuthFilter

Code:

```
// Public URLs that don't require authentication
private static final String[] PUBLIC_URLS = {
    "/login",
    "/logout",
    ".css",
    ".js",
    ".png",
    ".jpg",
    ".jpeg",
    ".gif"
};

@Override
public void init(FilterConfig filterConfig) throws ServletException {
    System.out.println("AuthFilter initialized");
}

@Override
public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain)
    throws IOException, ServletException {

    HttpServletRequest httpRequest = (HttpServletRequest) request;
    HttpServletResponse httpResponse = (HttpServletResponse) response;

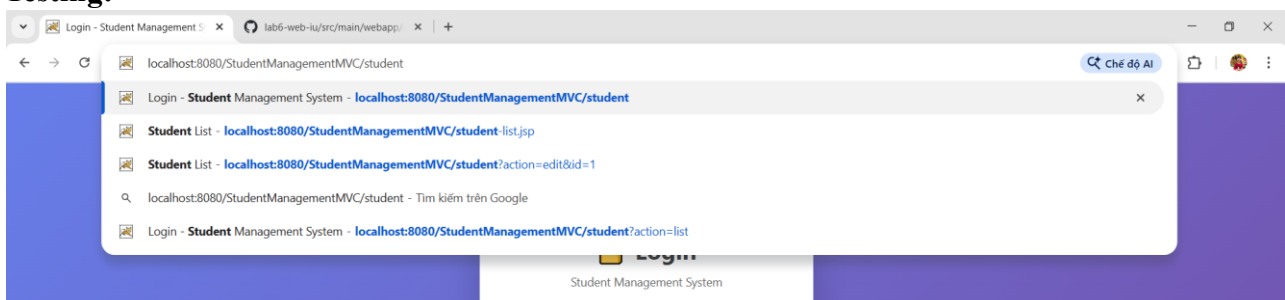
    String requestURI = httpRequest.getRequestURI();
    String contextPath = httpRequest.getContextPath();
    String path = requestURI.substring(contextPath.length());

    // Check if this is a public URL
    if (isPublicUrl(path)) {
        // Allow access to public URLs
        chain.doFilter(request, response);
        return;
    }

    // Check if user is logged in
    HttpSession session = httpRequest.getSession(false);
    boolean isLoggedIn = (session != null && session.getAttribute("user") != null);

    if (isLoggedIn) {
        // User is logged in, allow access
        chain.doFilter(request, response);
    } else {
        // User not logged in, redirect to login
        String loginURL = contextPath + "/login";
        httpResponse.sendRedirect(loginURL);
    }
}
```

Testing:



Try accessing /student without login

Redirect to /login

Student Management System

Welcome, John Doe user
[Dashboard](#) [Logout](#)

Student List

Search by name or student code...

Filter by Major:

ID	Student Code	Full Name	Email	Major
29	SV017	Jannik Sinner	janik@email.com	Software Engineering
26	SV001	John Smith	john.smith@email.com	Computer Science
22	SV016	John Doe	john@email.com	Computer Science
21	SV015	Lucas Tran	lucas.tran@email.com	Computer Science
20	SV014	Charlotte Do	charlotte.do@email.com	Software Engineering
19	SV013	Benjamin Le	benjamin.le@email.com	Data Science
18	SV012	Mia Vo	mia.vo@email.com	Information Technology
17	SV011	William Hoang	william.h@email.com	Computer Science
16	SV010	Ava Pham	ava.pham@email.com	Software Engineering
15	SV009	Daniel Kim	daniel.k@email.com	Data Science

Showing page 1 of 2

Access /student with login successfully

Student Management System

Welcome,
[Dashboard](#) [Logout](#)

Student List

Search by name or student code...

Filter by Major:

ID	Student Code	Full Name	Email	Major
29	SV017	Jannik Sinner	janik@email.com	Software Engineering
26	SV001	John Smith	john.smith@email.com	Computer Science
22	SV016	John Doe	john@email.com	Computer Science
21	SV015	Lucas Tran	lucas.tran@email.com	Computer Science
20	SV014	Charlotte Do	charlotte.do@email.com	Software Engineering
19	SV013	Benjamin Le	benjamin.le@email.com	Data Science
18	SV012	Mia Vo	mia.vo@email.com	Information Technology
17	SV011	William Hoang	william.h@email.com	Computer Science
16	SV010	Ava Pham	ava.pham@email.com	Software Engineering
15	SV009	Daniel Kim	daniel.k@email.com	Data Science

Access static file (student-list.jsp) without login

Explanation about the code flow of authentication filter

1. Browser → sends any GET/POST request such as /student?action=list, /student-list.jsp and send it to the server (AuthFilter.java).
2. The server (AuthFilter.java file) receives request first because WebFilter(urlPatterns = {"/*"}) intercepts every request before it reaches servlets or JSPs.
3. AuthFilter.java file extracts the request path from requestURI.
4. AuthFilter checks that is the request public? Checks if path matches any public URL: (/login, .css, .js, .png, etc.)
 - If the request is public, it skips the authentication and allows without checking session. Then, continue to servlet/JSP or static file.
 - If the request is not public, it must check authentication.
5. AuthFilter checks session:
 - If session exists AND session.getAttribute("user") is not null → user is logged in
 - If not logged in → redirect to /login → STOP FLOW here
6. If allowed → AuthFilter calls next filter / servlet: chain.doFilter() → request continues into controller or servlet or DAO.

EXERCISE 6: ADMIN AUTHORIZATION FILTER

Code:

```
// Admin-only actions
private static final String[] ADMIN_ACTIONS = {
    "new",
    "insert",
    "edit",
    "update",
    "delete"
};

@Override
public void init(FilterConfig filterConfig) throws ServletException {
    System.out.println("AdminFilter initialized");
}

@Override
public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain)
    throws IOException, ServletException {

    HttpServletRequest httpRequest = (HttpServletRequest) request;
    HttpServletResponse httpResponse = (HttpServletResponse) response;

    String action = httpRequest.getParameter("action");

    // Check if this action requires admin role
    if (isAdminAction(action)) {
        HttpSession session = httpRequest.getSession(false);

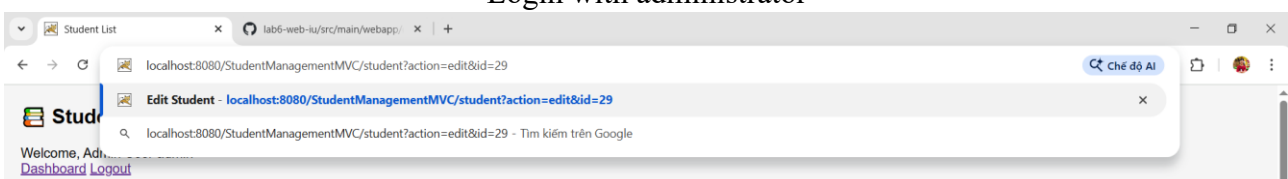
        if (session != null) {
            User user = (User) session.getAttribute("user");

            if (user != null && user.isAdmin()) {
                // User is admin, allow access
                chain.doFilter(request, response);
            } else {
                // User is not admin, deny access
                httpResponse.sendRedirect(httpRequest.getContextPath() +
                    "/student?action=list&error=Access denied. Admin privileges required.");
            }
        } else {
            // No session, redirect to login
            httpResponse.sendRedirect(httpRequest.getContextPath() + "/login");
        }
    } else {
        // Not an admin action, allow access
        chain.doFilter(request, response);
    }
}
```

Testing:



Login with administrator



Accessing /student?action=edit&id=29

Edit Student

Student Code:
SV017

Full Name:
Jannik Sinner

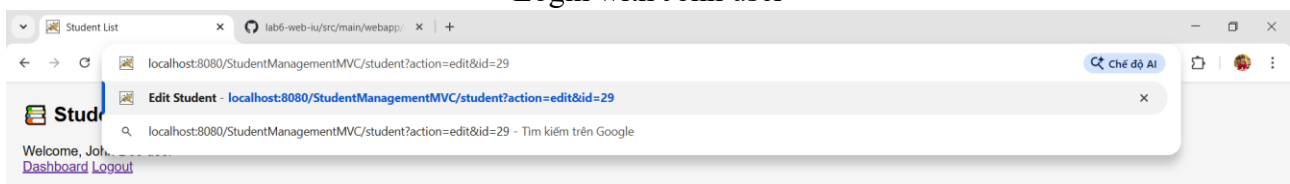
Email:
janik@email.com

Major:
Software Engineering

The edit form for administrator



Login with John user



Accessing /student?action=edit&id=29



The error message that John user cannot edit information

Explanation about the code flow of admin authorization filter:

1. Browse → sends request to /student such as /student?action=list, /student?action=new, etc, and sends it to the server (AdminFilter.java file).
2. The server (AdminFilter.java file) receives request before the servlet because of WebFilter(filterName = "AdminFilter", urlPatterns = {"/student"}). Therefore, it intercepts only StudentController, not CSS or other paths.
3. AdminFilter.java file reads the action parameter which is sent by browser.
4. Filter checks that is this an admin action?
5. If action is NOT admin action, skip filter for example /student?action=list and call chain.doFilter(request, response) to continue.
6. If action is admin action → filter checks session
 - If session == null which means that user is NOT logged in. Therefore, it redirect to /login and stop.
 - If user is null → Redirect to login.

- If user exists → Continue next condition
7. Filter checks if user is admin
- If user.isAdmin() = true → allow request
 - If user.isAdmin() = false → block request
 - If NOT admin → redirect: /student?action=list&error=Access denied. Admin privileges required.
8. If the user is admin, then the request continues to the server (StudentController.java file):
chain.doFilter(request, response);

EXERCISE 7: ROLE-BASED UI

Task 7.1: Update Student List View

Code:

```
<div class="navbar">
  <h2> Student Management System</h2>
  <div class="navbar-right">
    <div class="user-info">
      <span>Welcome, ${sessionScope.fullName}</span>
      <span class="role-badge role-${sessionScope.role}">
        ${sessionScope.role}
      </span>
    </div>
    <a href="dashboard" class="btn-nav">Dashboard</a>
    <a href="logout" class="btn-logout">Logout</a>
  </div>
</div>
```

Display role badge and username

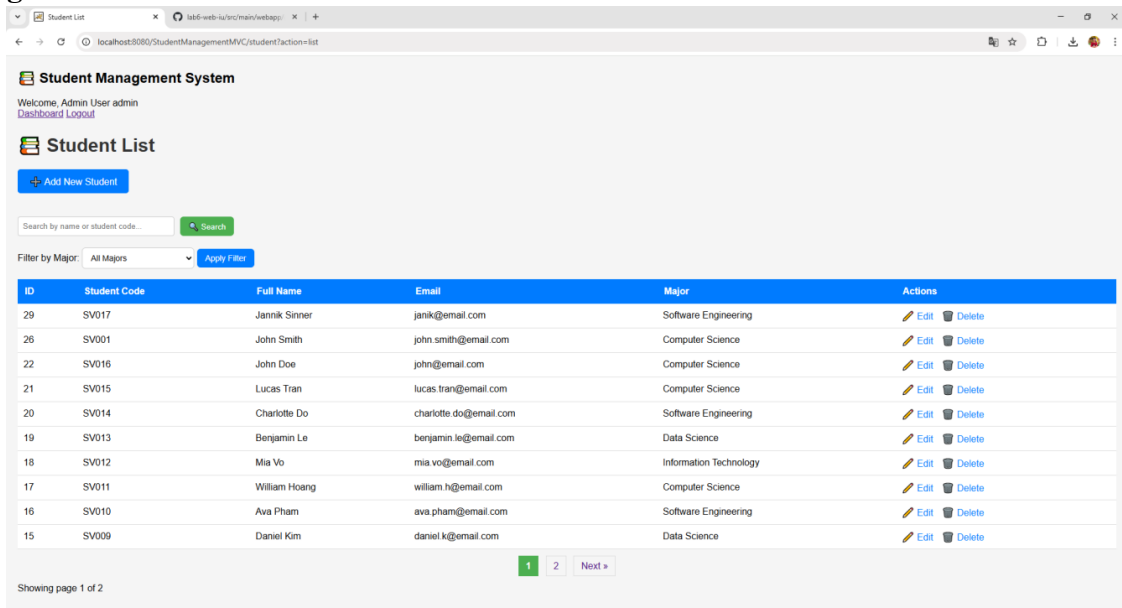
```
<div class="container">
  <h1> Student List</h1>
  <!-- Add button - Admin only -->
  <c:if test="${sessionScope.role eq 'admin'}">
    <div style="margin: 20px 0;">
      <a href="student?action=new" class="btn">+ Add New Student</a>
    </div>
  </c:if>
</div>
```

Modifying add button for administrator

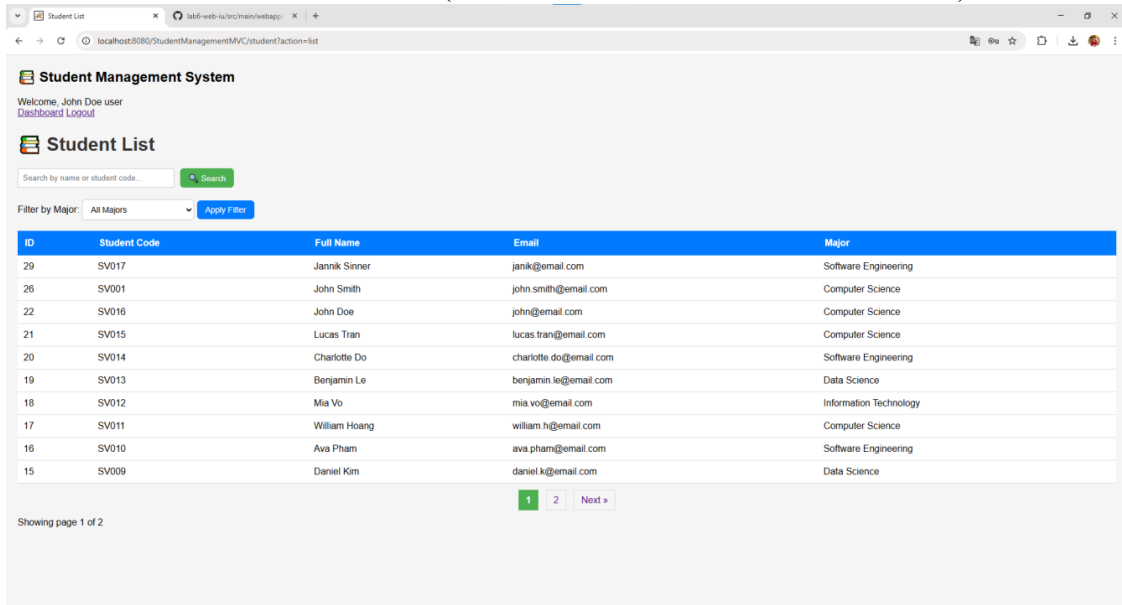
```
<c:if test="${sessionScope.role eq 'admin'}">
  <th>Actions</th>
</c:if>
</tr>
</thead>
<tbody>
  <c:forEach var="student" items="${students}">
    <tr>
      <td>${student.id}</td>
      <td>${student.studentCode}</td>
      <td>${student.fullName}</td>
      <td>${student.email != null ? student.email : 'N/A'}</td>
      <td>${student.major != null ? student.major : 'N/A'}</td>
      <!-- Action buttons - Admin only -->
      <c:if test="${sessionScope.role eq 'admin'}">
        <td>
          <a href="student?action=edit&id=${student.id}"
            class="action-link">✎ Edit</a>
          <a href="student?action=delete&id=${student.id}"
            class="action-link"
            onclick="return confirm('Delete this student?')">🗑 Delete</a>
        </td>
      </c:if>
    </tr>
  </c:forEach>
</tbody>
```

Edit and delete action for administrator

Testing:



Student list as admin (with Add/Edit/Delete buttons for action)



Student list as regular user (without Add/Edit/Delete)

Explanation about the code flow of role-based ui:

1. Browser → sends GET /student?action=list and sends it to the server.
2. The authentication and authorization (including AuthFilter.java and AdminFilter.java files) run first
 - AuthFilter checks if the user is logged in (sessionScope.user).
 - AdminFilter only restricts admin-only actions (new, insert, edit, update, delete).
 - For plain list, both filters allow the request to continue.
3. The server (StudentController.java file) → gets students and forwards to student-list.jsp
 - Controller sets attributes like students, currentPage, totalPages, sortBy, order, keyword, etc.
 - User info (fullName, role) is stored in session earlier at login.
 - Request is forwarded to: /student-list.jsp
4. JSP reads role & full name from session and renders navbar
 - If session role is "admin", badge shows "admin".
 - If session role is "user", badge shows the username.

5. JSP decides whether to show “Add New Student” button (admin-only)
 - If role == 'admin' → Add button is rendered
 - If not admin → that whole block is skipped → no Add button showing.
6. JSP decides whether to show the “Actions” column in the table header (admin-only)
 - If session role is “admin” → table has an extra “Actions” column.
 - If session role is “user” → table has no Actions column at all (only view data).
7. JSP loops through students and conditionally renders Edit/Delete buttons (admin-only)
 - If admin → each row has Edit and Delete buttons.
 - If not admin → that <td> isn’t rendered → user can only view, not modify.
8. JSP finishes rendering and sends final HTML back to browser.
 - Admin sees the list of students, “Add New Student”, an “Actions” column, and “Edit or Delete” buttons
 - Normal user sees the list of students, no “Add New Student” button, no “Actions” column, no “Edit or Delete” buttons → read-only UI
 - Backend filters (AdminFilter) still protect URLs even if someone manually types them.

EXERCISE 8: CHANGE PASSWORD (8 points)

Controller: `src/controller/ChangePasswordController.java`

Code:

```
protected void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {

    // Check login
    HttpSession session = request.getSession(false);
    if (session == null || session.getAttribute("user") == null) {
        response.sendRedirect("login");
        return;
    }

    request.getRequestDispatcher("/change-password.jsp").forward(request, response);
}

protected void doPost(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {

    HttpSession session = request.getSession(false);
    if (session == null || session.getAttribute("user") == null) {
        response.sendRedirect("login");
        return;
    }

    User user = (User) session.getAttribute("user");

    String currentPassword = request.getParameter("currentPassword");
    String newPassword = request.getParameter("newPassword");
    String confirmPassword = request.getParameter("confirmPassword");

    // 1. Check empty fields
    if (currentPassword == null || newPassword == null || confirmPassword == null ||
        currentPassword.trim().isEmpty() || newPassword.trim().isEmpty() || confirmPassword.trim().isEmpty()) {

        request.setAttribute("error", "All fields are required.");
        request.getRequestDispatcher("/change-password.jsp").forward(request, response);
        return;
    }

    // 2. Validate current password
    if (!BCrypt.checkpw(currentPassword, user.getPassword())) {
        request.setAttribute("error", "Current password is incorrect.");
        request.getRequestDispatcher("/change-password.jsp").forward(request, response);
        return;
    }

    // 3. Validate new password length
    if (newPassword.length() < 8) {
        request.setAttribute("error", "New password must be at least 8 characters long.");
        request.getRequestDispatcher("/change-password.jsp").forward(request, response);
        return;
    }
}
```

```

// 4. Confirm new password matches
if (!newPassword.equals(confirmPassword)) {
    request.setAttribute("error", "New password and confirm password do not match.");
    request.getRequestDispatcher("/change-password.jsp").forward(request, response);
    return;
}

// 5. Hash new password
String newHashedPassword = BCrypt.hashpw(newPassword, BCrypt.gensalt());

// 6. Update DB
boolean updated = userDAO.updatePassword(user.getId(), newHashedPassword);

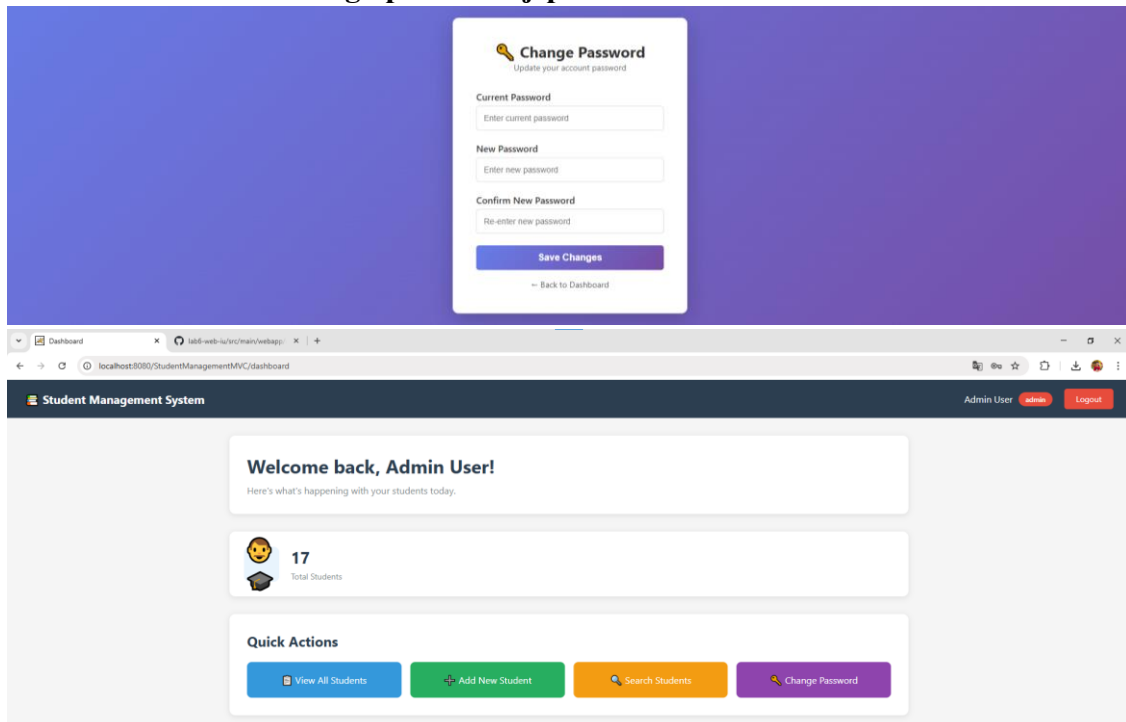
if (updated) {
    // Also update in session
    user.setPassword(newHashedPassword);
    session.setAttribute("user", user);

    request.setAttribute("success", "Password updated successfully!");
} else {
    request.setAttribute("error", "Something went wrong. Try again.");
}

request.getRequestDispatcher("/change-password.jsp").forward(request, response);
}

```

View: WebContent/views/change-password.jsp



The change password button in dashboard (just for testing the function change password)

DAO Method: Add to UserDAO.java

```

public boolean updatePassword(int userId, String newHashedPassword) {
    try (Connection conn = getConnection();
        PreparedStatement stmt = conn.prepareStatement(SQL_UPDATE_PASSWORD)) {

        stmt.setString(1, newHashedPassword);
        stmt.setInt(2, userId);

        int rowsUpdated = stmt.executeUpdate();

        return rowsUpdated > 0; // true if update successful
    } catch (Exception e) {
        e.printStackTrace();
        return false;
    }
}

public boolean checkPassword(String plainPassword, String hashedPassword) {
    return BCrypt.checkpw(plainPassword, hashedPassword);
}

```

Explanation about the code flow of change password:

A. Open the change password page (GET)

1. Browser → GET /change-password: User clicks “Change Password” in the app (or manually enters URL) and sends it to the server (ChangePasswordController.java file). For testing, I just put that in dashboard, which is only for administrator.
2. The server (ChangePasswordController.java file) receives the request via .doGet() method.
 - If no session or no user in session, which means that user is not logged in. Therefore, it redirects to login page and STOP.
 - If the user logged in → continue.
3. Controller forwards to JSP form
4. JSP renders the change-password form
 - Shows error/success message if they exist ({error}, {success})
 - Displays three password fields: currentPassword, newPassword, confirmPassword.
5. Form submits with POST method.
6. Browser displays the “Change Password” page ready for input.

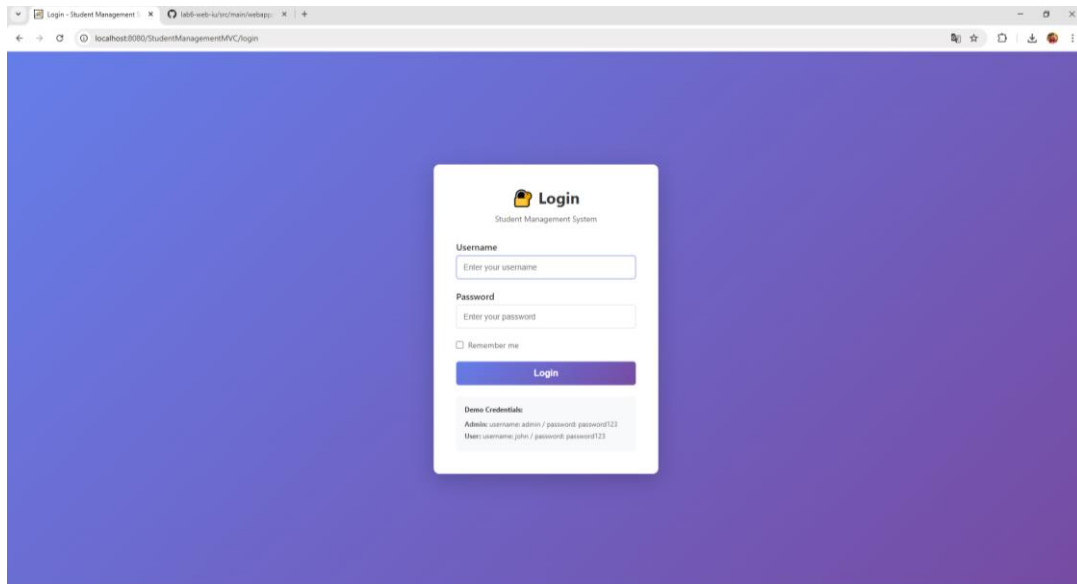
B. User submits the change password form (POST)

1. Browser → POST /change-password.
 - The user fills in all information (currentPassword, newPassword, confirmPassword). When the user clicks the “Save changes” button, a POST request is sent to controller (ChangePasswordController.java file).
2. ChangePasswordController.doPost() receives the request: It checks login:
 - If the user not logged in → redirect to login page, stop.
 - If the user logged in. Then, it gets the User from session:
User user = (User) session.getAttribute("user");
3. Controller reads form fields:

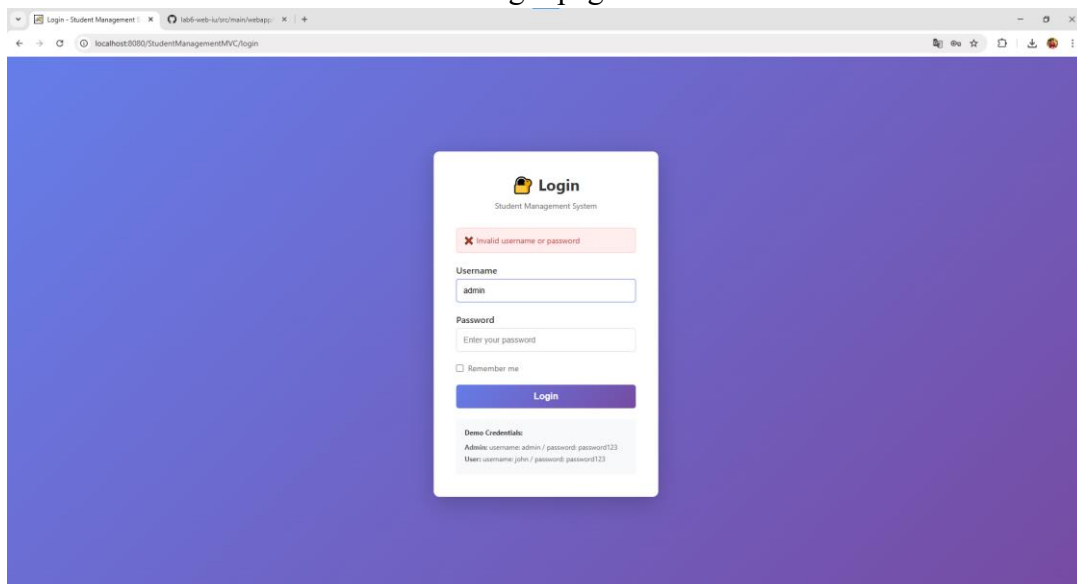
```
String currentPassword = request.getParameter("currentPassword");
String newPassword = request.getParameter("newPassword");
String confirmPassword = request.getParameter("confirmPassword");
```

Controller runs validation BEFORE doing anything else
4. Check empty fields of currentPassword, newPassword, and confirmPassword
 - If any field is empty → set error + forward back to JSP (no redirect).
 - JSP shows an error message that “All fields are required.”.
5. Check current password is correct (BCrypt)
 - user.getPassword() is the stored hashed password.
 - BCrypt.checkpw() compares plain currentPassword with hashed one.
 - If it doesn’t match → set error, forward back to JSP, and shows that error message.
6. Check new password strength (length): If it is too short, print an error message that password requires at least 8 characters.
7. Confirm new password matches: If it is mismatch, print an error message.
8. Hash the new password with Bcrypt
 - Never store plain text password
 - gensalt() generates a random salt, then hashpw() hashes new password
9. Update password in database via UserDao:
boolean updated = userDao.updatePassword(user.getId(), newHashedPassword);
10. If database update is successful: Session’s user object is updated with the new hashed password. A success message is set “Password updated successfully!”.
11. If database update is unsuccessful: Error message is set “Something went wrong. Try again.”.
12. Forward back to the same JSP, which then displays the message at the top since I just implemented it for testing.

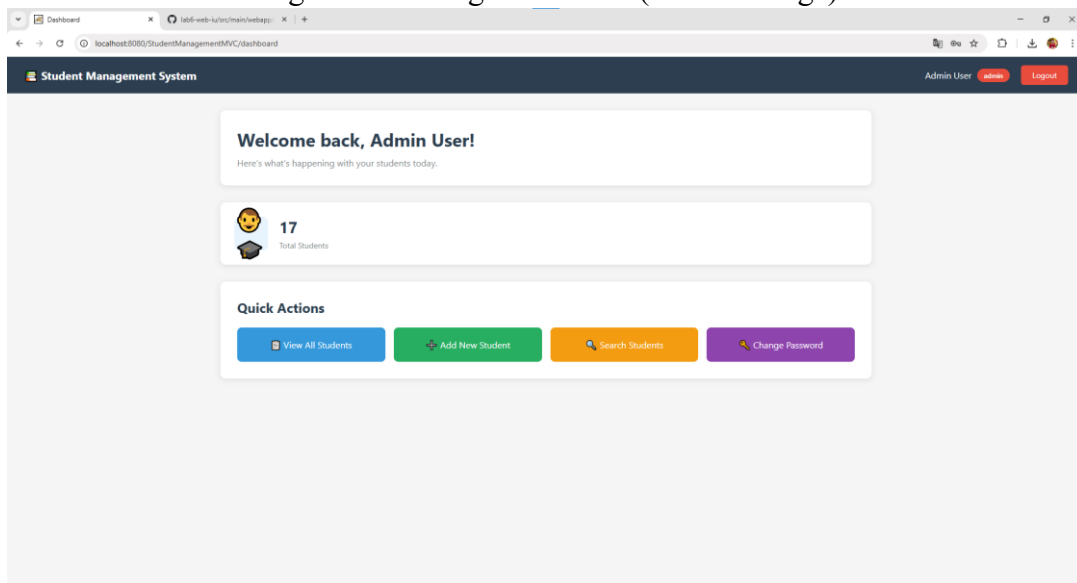
Screenshots:



Login page



Login with wrong credentials (error message)



Dashboard after logging in with administrator account

Student Management System
Welcome, Admin User admin
[Dashboard](#) [Logout](#)

Student List

[Add New Student](#)

Search by name or student code... [Search](#)

Filter by Major: All Majors [Apply Filter](#)

ID	Student Code	Full Name	Email	Major	Actions
29	SV017	Jannik Sinner	janik@email.com	Software Engineering	Edit Delete
26	SV001	John Smith	john.smith@email.com	Computer Science	Edit Delete
22	SV016	John Doe	john@email.com	Computer Science	Edit Delete
21	SV015	Lucas Tran	lucas.tran@email.com	Computer Science	Edit Delete
20	SV014	Charlotte Do	charlotte.do@email.com	Software Engineering	Edit Delete
19	SV013	Benjamin Le	benjamin.le@email.com	Data Science	Edit Delete
18	SV012	Mia Vo	mia.vo@email.com	Information Technology	Edit Delete
17	SV011	William Hoang	william.h@email.com	Computer Science	Edit Delete
16	SV010	Ava Pham	ava.pham@email.com	Software Engineering	Edit Delete
15	SV009	Daniel Kim	daniel.k@email.com	Data Science	Edit Delete

Showing page 1 of 2

Student list as admin (with Add/Edit/Delete buttons)

Student Management System
Welcome, John Doe user
[Dashboard](#) [Logout](#)

Student List

Search by name or student code... [Search](#)

Filter by Major: All Majors [Apply Filter](#)

ID	Student Code	Full Name	Email	Major	
29	SV017	Jannik Sinner	janik@email.com	Software Engineering	
26	SV001	John Smith	john.smith@email.com	Computer Science	
22	SV016	John Doe	john@email.com	Computer Science	
21	SV015	Lucas Tran	lucas.tran@email.com	Computer Science	
20	SV014	Charlotte Do	charlotte.do@email.com	Software Engineering	
19	SV013	Benjamin Le	benjamin.le@email.com	Data Science	
18	SV012	Mia Vo	mia.vo@email.com	Information Technology	
17	SV011	William Hoang	william.h@email.com	Computer Science	
16	SV010	Ava Pham	ava.pham@email.com	Software Engineering	
15	SV009	Daniel Kim	daniel.k@email.com	Data Science	

Showing page 1 of 2

Student list as regular user (without Edit/Delete)

login - Student Management System

localhost:8080/StudentManagementMVC/student?action=list

Student List - localhost:8080/StudentManagementMVC/student?action=list

login - Student Management System

localhost:8080/StudentManagementMVC/login

Accessing protected page without login (redirect)

Student List

localhost:8080/StudentManagementMVC/student?action=edit&id=1

HTTP Status 404 - Not Found - localhost:8080/StudentManagementMVC/student?action=edit&id=1

localhost:8080/StudentManagementMVC/student?action=edit&id=1 - Tìm kiếm trên Google

Student List

Welcome, John Doe user
[Dashboard](#) [Logout](#)

Student List

Welcome, John Doe user
[Dashboard](#) [Logout](#)

Student List

Access denied. Admin privileges required.

Search by name or student code...

Filter by Major:

ID	Student Code	Full Name	Email	Major
29	SV017	Jannik Sinner	janik@email.com	Software Engineering
26	SV001	John Smith	john.smith@email.com	Computer Science
22	SV016	John Doe	john@email.com	Computer Science
21	SV015	Lucas Tran	lucas.tran@email.com	Computer Science
20	SV014	Charlotte Do	charlotte.do@email.com	Software Engineering
19	SV013	Benjamin Le	benjamin.le@email.com	Data Science
18	SV012	Mia Vo	mia.vo@email.com	Information Technology
17	SV011	William Hoang	william.h@email.com	Computer Science
16	SV010	Ava Pham	ava.pham@email.com	Software Engineering
15	SV009	Daniel Kim	daniel.k@email.com	Data Science

Showing page 1 of 2

1 2 Next >

Non-admin trying admin action (blocked)

Login - Student Management

localhost:8080/StudentManagementMVC/login?message=You%20have%20been%20logged%20out%20successfully

Login

Student Management System

☒ You have been logged out successfully

Username

Password

☐ Remember me

Demo Credentials:
 Admin: username: admin / password: password123
 User: username: john / password: password123

Logout success message